Python Grundlagen

Syntax und Kontrollstrukturen

Grundlegende Python-Syntax, Bedingungen,
Schleifen,
`break`, `pass` und `try-except`

Inhaltsverzeichnis

1 Grundlegende Python-Syntax **2**
1.1 Beispiel 2

2 Bedingungen mit if **2**
2.1 Beispiel 2

3 for-Schleife **2**
3.1 Beispiel 3

4 while-Schleife **3**
4.1 Beispiel 3

5 break **3**
5.1 Beispiel 3

6 pass **3**
6.1 Beispiel 3

7 try-except **4**
7.1 Beispiel 4

1 Grundlegende Python-Syntax

Python besitzt eine einfache und übersichtliche Syntax. Im Gegensatz zu Java oder C# müssen Variablen nicht mit einem Datentyp deklariert werden. Der Datentyp wird automatisch anhand des zugewiesenen Wertes erkannt.

Eine wichtige Besonderheit von Python ist die Verwendung von Einrückungen. Sie bestimmen, welche Anweisungen zu einem Codeblock gehören. Geschweifte Klammern wie in Java oder C# werden daher nicht benötigt.

Mit `print()` können Werte oder Texte auf der Konsole ausgegeben werden.

1.1 Beispiel

```
name = "Mathias"
alter = 18

print("Name:", name)
print("Alter:", alter)
```

2 Bedingungen mit if

Mit `if` können Bedingungen überprüft werden. Ist eine Bedingung wahr, wird der eingerückte Code ausgeführt.

Mit `elif` können weitere Bedingungen überprüft werden. `else` wird ausgeführt, wenn keine der vorherigen Bedingungen zutrifft.

2.1 Beispiel

```
alter = 18

if alter >= 18:
    print("Du bist volljaehrig.")
elif alter >= 16:
    print("Du bist zwischen 16 und 17 Jahre alt.")
else:
    print("Du bist unter 16 Jahre alt.")
```

3 for-Schleife

Eine `for`-Schleife wird verwendet, um einen Codeblock mehrmals auszuführen. Häufig wird dabei `range()` verwendet, um einen bestimmten Zahlenbereich zu durchlaufen.

Bei `range(5)` beginnt die Zählung standardmäßig bei 0 und endet vor der Zahl 5.

3.1 Beispiel

```
for i in range(5):
    print("Zahl:", i)
```

4 while-Schleife

Eine `while`-Schleife wiederholt einen Codeblock so lange, wie eine bestimmte Bedingung wahr ist.

Die Bedingung muss sich innerhalb der Schleife irgendwann verändern. Andernfalls kann eine Endlosschleife entstehen.

4.1 Beispiel

```
zahl = 0

while zahl < 5:
    print("Zahl:", zahl)
    zahl += 1
```

5 break

Mit `break` kann eine Schleife vorzeitig beendet werden. Sobald Python auf `break` trifft, wird die aktuelle Schleife sofort verlassen.

5.1 Beispiel

```
for i in range(10):
    if i == 5:
        break

    print(i)
```

6 pass

Die Anweisung `pass` führt keine Aktion aus. Sie dient als Platzhalter, wenn ein Codeblock syntaktisch benötigt wird, aber momentan noch keinen Inhalt besitzen soll.

Dies kann beispielsweise beim Erstellen einer Funktion hilfreich sein, deren Inhalt erst später programmiert wird.

6.1 Beispiel

```python
alter = 18

if alter >= 19:
    pass
else:
    print("Du bist noch nicht volljaehrig.")

print("Das Programm laeuft weiter.")
```

7 try-except

Mit `try-except` können Fehler während der Programmausführung abgefangen und behandelt werden.

Der Code, bei dem ein Fehler auftreten könnte, wird in den `try`-Block geschrieben. Mit `except` wird festgelegt, wie auf einen bestimmten Fehler reagiert werden soll.

7.1 Beispiel

```python
try:
    zahl = int(input("Gib eine Zahl ein: "))
    ergebnis = 10 / zahl
    print("Ergebnis:", ergebnis)

except ValueError:
    print("Fehler: Du musst eine Zahl eingeben.")

except ZeroDivisionError:
    print("Fehler: Division durch 0 ist nicht moeglich.")
```